

两种形式并无差别, 纯粹只是小括号的摆放位置不同而已。我称此二种形式为“旧式转型” (*old-style casts*)。

C++ 还提供四种新式转型 (常常被称为 *new-style* 或 *C++-style casts*) :

```
const_cast<T>( expression )  
dynamic_cast<T>( expression )  
reinterpret_cast<T>( expression )  
static_cast<T>( expression )
```

各有不同的目的:

- `const_cast` 通常被用来将对象的常量性转除 (cast away the constness)。它也是唯一有此能力的 C++-style 转型操作符。
- `dynamic_cast` 主要用来执行“安全向下转型” (safe downcasting), 也就是用来决定某对象是否归属继承体系中的某个类型。它是唯一无法由旧式语法执行的动作, 也是唯一可能耗费重大运行成本的转型动作 (稍后细谈)。
- `reinterpret_cast` 意图执行低级转型, 实际动作 (及结果) 可能取决于编译器, 这也就表示它不可移植。例如将一个 *pointer to int* 转型为一个 `int`。这一类转型在低级代码以外很少见。本书只使用一次, 那是在讨论如何针对原始内存 (raw memory) 写出一个调试用的分配器 (debugging allocator) 时, 见条款 50。
- `static_cast` 用来强迫隐式转换 (implicit conversions), 例如将 `non-const` 对象转为 `const` 对象 (就像条款 3 所为), 或将 `int` 转为 `double` 等等。它也可以用来执行上述多种转换的反向转换, 例如将 `void*` 指针转为 `typed` 指针, 将 *pointer-to-base* 转为 *pointer-to-derived*。但它无法将 `const` 转为 `non-const`——这个只有 `const_cast` 才办得到。

旧式转型仍然合法, 但新式转型较受欢迎。原因是: 第一, 它们很容易在代码中被辨识出来 (不论是人工辨识或使用工具如 `grep`), 因而得以简化“找出类型系统在哪个地点被破坏”的过程。第二, 各转型动作的目标愈窄化, 编译器愈可能诊断出错误的运用。举个例子, 如果你打算将常量性 (constness) 去掉, 除非使用新式转型中的 `const_cast` 否则无法通过编译。

我唯一使用旧式转型的时机是, 当我要调用一个 `explicit` 构造函数将一个对象传递给一个函数时。例如:

```

class Widget {
public:
    explicit Widget(int size);
    ...
};

void doSomeWork(const Widget& w);
doSomeWork(Widget(15));           //以一个 int 加上“函数风格”的
                                   //转型动作创建一个 Widget。
doSomeWork(static_cast<Widget>(15)); //以一个 int 加上“C++ 风格”的
                                   //转型动作创建一个 Widget。

```

从某个角度来说，蓄意的“对象生成”动作感觉不怎么像“转型”，所以我很可能使用函数风格的转型动作而不使用 `static_cast`。但我要再说一次，当我们写下一段日后出错导致“核心倾印”（core dump）的代码时，撰写之时我们往往“觉得”通情达理，所以或许最好是忽略你的感觉，始终理智地使用新式转型。

许多程序员相信，转型其实什么都没做，只是告诉编译器把某种类型视为另一种类型。这是错误的观念。任何一个类型转换（不论是通过转型操作而进行的显式转换，或通过编译器完成的隐式转换）往往真的令编译器编译出运行期间执行的码。例如在这段程序中：

```

int x, y;
...
double d = static_cast<double>(x)/y;           //x 除以 y，使用浮点数除法

```

将 `int x` 转型为 `double` 几乎肯定会产生一些代码，因为在大部分计算器体系结构中，`int` 的底层表述不同于 `double` 的底层表述。这或许不会让你惊讶，但下面这个例子就有可能让你稍微睁大眼睛了：

```

class Base { ... };
class Derived: public Base { ... };
Derived d;
Base* pb = &d;           //隐喻地将 Derived* 转换为 Base*

```

这里我们不过是建立一个 `base class` 指针指向一个 `derived class` 对象，但有时候上述的两个指针值并不相同。这种情况下会有个偏移量（offset）在运行期被施行于 `Derived*` 指针身上，用以取得正确的 `Base*` 指针值。

上个例子表明，单一对象（例如一个类型为 `Derived` 的对象）可能拥有一个以上的地址（例如“以 `Base*` 指向它”时的地址和“以 `Derived*` 指向它”时的地址。C 不可能发生这种事，Java 不可能发生这种事，C# 也不可能发生这种事。但 C++ 可能！实际上一旦使用多重继承，这事几乎一直发生着。即使在单一继承中也可能发

生。虽然这还有其他意涵,但至少意味你通常应该避免做出“对象在 C++ 中如何如何布局”的假设。当然更不该以此假设为基础执行任何转型动作。例如,将对象地址转型为 `char*` 指针然后在它们身上进行指针算术,几乎总是会导致无定义(不明确)行为。

但请注意,我说的是有时候需要一个偏移量。对象的布局方式和它们的地址计算方式随编译器的不同而不同,那意味“由于知道对象如何布局”而设计的转型,在某一平台行得通,在其他平台并不一定行得通。这个世界有许多悲惨的程序员,他们历经千辛万苦才学到这堂课。

另一件关于转型的有趣事情是:我们很容易写出某些似是而非的代码(在其他语言中也许真是对的)。例如许多应用框架(application frameworks)都要求 derived classes 内的 virtual 函数代码的第一个动作就先调用 base class 的对应函数。假设我们有个 Window base class 和一个 SpecialWindow derived class,两者都定义了 virtual 函数 onResize。进一步假设 SpecialWindow 的 onResize 函数被要求首先调用 Window 的 onResize。下面是实现方式之一,它看起来对,但实际上错:

```
class Window {                                //base class
public:
    virtual void onResize( ) { ... }          //base onResize 实现代码
    ...
};

class SpecialWindow: public Window {          //derived class
public:
    virtual void onResize( ) {                //derived onResize 实现代码
        static_cast<Window>(*this).onResize(); //将*this 转型为 Window,
                                                //然后调用其 onResize;
                                                //这不可行!
        ... //这里进行 SpecialWindow 专属行为。
    }
    ...
};
```

我在代码中强调了转型动作(那是个新式转型,但若使用旧式转型也不能改变以下事实)。一如你所预期,这段程序将*this 转型为 Window,对函数 onResize 的调用也因此调用了 Window::onResize。但恐怕你没想到,它调用的并不是当前对象上的函数,而是稍早转型动作所建立的一个“*this 对象之 base class 成分”的暂时副本身上的 onResize! (译注:函数就是函数,成员函数只有一份,“调用起哪个对象身上的函数”有什么关系呢?关键在于成员函数都有个隐藏的 this 指

针，会因此影响成员函数操作的数据。)再说一次，上述代码并非在当前对象身上调用 `Window::onResize` 之后又在该对象身上执行 `SpecialWindow` 专属动作。不，它是在“当前对象之 `base class` 成分”的副本上调用 `Window::onResize`，然后在当前对象身上执行 `SpecialWindow` 专属动作。如果 `Window::onResize` 修改了对象内容（不能说没有可能性，因为 `onResize` 是个 `non-const` 成员函数），当前对象其实没被改动，改动的是副本。然而 `SpecialWindow::onResize` 内如果也修改对象，当前对象真的会被改动。这使当前对象进入一种“伤残”状态：其 `base class` 成分的更改没有落实，而 `derived class` 成分的更改倒是落实了。

解决之道是拿掉转型动作，代之以你真正想说的话。你并不想哄骗编译器将 `*this` 视为一个 `base class` 对象，你只是想调用 `base class` 版本的 `onResize` 函数，令它作用于当前对象身上。所以请这么写：

```
class SpecialWindow: public Window {
public:
    virtual void onResize() {
        Window::onResize();    //调用 Window::onResize 作用于*this 身上
        ...
    }
    ...
};
```

这个例子也说明，如果你发现你自己打算转型，那活脱是个警告信号：你可能正将局面发展至错误的方向上。如果你用的是 `dynamic_cast` 更是如此。

在探究 `dynamic_cast` 设计意涵之前，值得注意的是，`dynamic_cast` 的许多实现版本执行速度相当慢。例如至少有一个很普遍的实现版本基于“`class` 名称之字符串比较”，如果你在四层深的单继承体系内的某个对象身上执行 `dynamic_cast`，刚才说的那个实现版本所提供的每一次 `dynamic_cast` 可能会耗用多达四次的 `strcmp` 调用，用以比较 `class` 名称。深度继承或多重继承的成本更高！某些实现版本这样做有其原因（它们必须支持动态连接）。然而我还是要强调，除了对一般转型保持机敏与猜疑，更应该注重效率的代码中对 `dynamic_casts` 保持机敏与猜疑。

之所以需要 `dynamic_cast`，通常是因为你想在一个你认定为 `derived class` 对象身上执行 `derived class` 操作函数，但你的手上却只有一个“指向 `base`”的 `pointer` 或 `reference`，你只能靠它们来处理对象。有两个一般性做法可以避免这个问题。

第一, 使用容器并在其中存储直接指向 **derived class** 对象的指针 (通常是智能指针, 见条款 13), 如此便消除了“通过 **base class** 接口处理对象”的需要。假设先前的 **Window/SpecialWindow** 继承体系中只有 **SpecialWindows** 才支持闪烁效果, 试着不要这样做:

```
class Window { ... };
class SpecialWindow: public Window {
public:
    void blink();
    ...
};

typedef                                //关于 tr1::shared_ptr
std::vector<std::tr1::shared_ptr<Window> > VPW; //见条款 13.
VPW winPtrs;
...
for (VPW::iterator iter = winPtrs.begin();    //不希望使用
     iter != winPtrs.end(); ++iter) {         //dynamic_cast.
    if (SpecialWindow * psw = dynamic_cast<SpecialWindow * >(iter->get()))
        psw->blink();
}
```

应该改而这样做:

```
typedef std::vector<std::tr1::shared_ptr<SpecialWindow> > VPSW;
VPSW winPtrs;
...
for ( VPSW::iterator iter = winPtrs.begin();    //这样写比较好,
     iter != winPtrs.end();                     //不使用 dynamic_cast
     ++iter)
    (*iter)->blink();
```

当然啦, 这种做法使你无法在同一个容器内存储指针“指向所有可能之各种 **Window** 派生类”。如果真要处理多种窗口类型, 你可能需要多个容器, 它们都必须具备类型安全性 (**type-safe**)。

另一种做法可让你通过 **base class** 接口处理“所有可能之各种 **Window** 派生类”, 那就是在 **base class** 内提供 **virtual** 函数做你想对各个 **Window** 派生类做的事。举个例子, 虽然只有 **SpecialWindows** 可以闪烁, 但或许将闪烁函数声明于 **base class** 内并提供一份“什么也没做”的缺省实现码是有意义的:

```

class Window {
public:
    virtual void blink() { }           //缺省实现代码“什么也没做”；
    ...                               //条款 34 告诉你为什么
};                                   // 缺省实现代码可能是个馊主意。

class SpecialWindow: public Window {
public:
    virtual void blink() { ... };      //在此 class 内，
    ...                               //blink 做某些事。
};

typedef std::vector<std::tr1::shared_ptr<Window> > VPW;
VPW winPtrs;                         //容器，内含指针，指向
...                                  // 所有可能的 Window 类型。
for (VPW::iterator iter = winPtrs.begin( );
    iter != winPtrs.end();
    ++iter)                          //注意，这里没有
    (*iter)->blink();                 // dynamic_cast。

```

不论哪一种写法——“使用类型安全容器”或“将 virtual 函数往继承体系上方移动”——都并非放之四海皆准，但在许多情况下它们都提供一个可行的 dynamic_cast 替代方案。当它们有此功效时，你应该欣然拥抱它们。

绝对必须避免的一件事是所谓的“连串 (cascading) dynamic_casts”，也就是看起来像这样的东西：

```

class Window { ... };
...                                //derived classes 定义在这里
typedef std::vector<std::tr1::shared_ptr<Window> > VPW;
VPW winPtrs;
...
for (VPW::iterator iter = winPtrs.begin( );
    iter != winPtrs.end(); ++iter)
{
    if (SpecialWindow1 * psw1 =
        dynamic_cast<SpecialWindow1*>(iter->get())) { ... }
    else if (SpecialWindow2 * psw2 =
        dynamic_cast<SpecialWindow2*>(iter->get())) { ... }
    else if (SpecialWindow3 * psw3 =
        dynamic_cast<SpecialWindow3*>(iter->get())) { ... }
    ...
}

```

这样产生出来的代码又大又慢, 而且基础不稳, 因为每次 window class 继承体系一有改变, 所有这一类代码都必须再次检阅看看是否需要修改。例如一旦加入新的 derived class, 或许上述连串判断中需要加入新的条件分支。这样的代码应该总是以某些“基于 virtual 函数调用”的东西取而代之。

优良的 C++ 代码很少使用转型, 但若说要完全摆脱它们又太过不切实际。例如 p.118 从 int 转型为 double 就是转型的一个通情达理的使用, 虽然它并非绝对必要(那段代码可以重新写过, 声明一个类型为 double 的新变量并以 x 值初始化)。就像面对众多蹊跷可疑的构造函数一样, 我们应该尽可能隔离转型动作, 通常是把它隐藏在某个函数内, 函数的接口会保护调用者不受函数内部任何肮脏龌龊的动作影响。

请记住

- 如果可以, 尽量避免转型, 特别是在注重效率的代码中避免 dynamic_casts。如果有个设计需要转型动作, 试着发展无需转型的替代设计。
- 如果转型是必要的, 试着将它隐藏于某个函数背后。客户随后可以调用该函数, 而不需将转型放进他们自己的代码内。
- 宁可使用 C++-style (新式) 转型, 不要使用旧式转型。前者很容易辨识出来, 而且也比较有着分门别类的职掌。

条款 28: 避免返回 handles 指向对象内部成分

Avoid returning "handles" to object internals.

假设你的程序涉及矩形。每个矩形由其左上角和右下角表示。为了让一个 Rectangle 对象尽可能小, 你可能会决定不把定义矩形的这些点存放在 Rectangle 对象内, 而是放在一个辅助的 struct 内再让 Rectangle 去指它:

```
class Point {           //这个 class 用来表述“点”
public:
    Point(int x, int y);
    ...
    void setX(int newVal);
    void setY(int newVal);
    ...
};
```